KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 6 Laboratuvar Föyü

FastAPI I: HTTP'yi Koda Dökmek — Etkinlik/Bilet Sisteminin İlk Sunucusunu Yazın

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: Python 3.12 (asgari) kurulu bir bilgisayar, bir metin editörü (VS Code önerilir), terminal/PowerShell erişimi, Hafta 1'den tanıdık `curl` — paketleri indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Geçen haftanın laboratuvarında oluşturduğunuz `Bilet[]` nesneleri yalnızca tarayıcı sekmenizin belleğinde yaşıyordu; sayfayı yenilediğinizde ya da bir arkadaşınız aynı siteyi açtığında bu biletleri hiç göremezdi — veri hiçbir zaman bilgisayarınızın dışına çıkmadı. Bugün iki saat sonra Etkinlik/Koltuk/Bilet için gerçek bir Python sunucusu (FastAPI) çalışır durumda olacak: iki farklı terminal penceresinden (iki "kullanıcı") gönderdiğiniz `curl` istekleri aynı depoyu okuyup yazacak, bir koltuğu iki kez satmaya çalıştığınızda sunucu bunu `409 Conflict` ile reddedecek ve tüm bu API'yi tek satır ek kod yazmadan `/docs` sayfasında gezilebilir hâle getireceksiniz — Etkinlik/Bilet Sisteminin ilk gerçek sunucusu bugün ortaya çıkacak.

1 • Ön-öğretim — Ön Bilgi

Bu föy Hafta 6 ders notunun dört ünitesine dayanır: Ünite 1 (İlk Uygulama, Yol/Sorgu/Gövde Parametreleri), Ünite 2 (Pydantic Modelleri ve Doğrulama), Ünite 3 (CRUD, Durum Kodları, Hata Cevapları) ve Ünite 4 (OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu Girişi). Hafta 1 Ünite 2–4'ün istemci-sunucu modelini, HTTP metot/durum kodu anatomisini ve `curl` deneyiminizi (bugün aynı `curl` komutları kendi sunucunuza gidecek) hazır bilgi olarak kullanacaksınız; Hafta 3 Ünite 3'ün `Etkinlik`/`Koltuk`/`Bilet` TS arayüzleri bugünkü Pydantic modellerinizin birebir karşılığıdır. Ön yüz (React) bu hafta dokunulmaz — `kod_ornekleri/hafta05/istemci/` klasörü değişmeden durur; bugün yazacağınız sunucu Hafta 7'de bu ön yüze bağlanacaktır. Veri bu hafta yalnızca sunucu sürecinin belleğinde yaşar — `--reload` dahil sunucuyu her yeniden başlattığınızda tüm veri kaybolur; kalıcılık (veritabanı) Hafta 8'in konusudur.

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi sunucu projenize **elle yazarak** tamamlayın, sonra terminaldeki/tarayıcıdaki çıktınızı *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta06/sunucu/` klasöründeki referans dosyalardan gelir — bunlar hedef/"doğru cevap" niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi dosyalarınıza yazarsınız, sonucu terminalde/tarayıcıda karşılaştırırsınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

Bu hafta yepyeni bir proje: `kod_ornekleri/hafta05/ istemci/`'nin aksine, bugün `sunucu/` adında, kendi başına çalışan, önceki haftaların hiçbir dosyasına bağımlı olmayan bir Python projesi kuracaksınız. `istemci/` bu hafta bu klasöre kopyalanmaz — ön yüz Hafta 7'de bu sunucuya

bağlanacaktır.

1 İlk Uygulama, Yol/Sorgu/Gövde Parametreleri

(bkz. Ders notu Hafta 6, Ünite 1 — İlk Uygulama, Yol/Sorgu/Gövde Parametreleri.)

Adım 1: Proje klasörünü ve sanal ortamı oluşturun

Ne yapacağız? `sunucu` adında bir klasör oluşturacak, içine Python 3.12 ile bir sanal ortam (`venv`) kuracak, etkinleştirecek ve iki bağımlılık dosyasını (`requirements.txt`, `requirements-dev.txt`) yazıp kuracaksınız.

Neden? Bir **sanal ortam**, bu projeye özel, diğer projelerden izole bir Python kurulumudur — farklı projeler farklı paket sürümlerine ihtiyaç duyabilir, küresel bir `pip install` bu ihtiyaçları çakıştırır (bir mutfakta birden fazla tarif hazırlıyorsanız, her tarifin malzemesini ayrı bir çekmecede tutmaya benzer). Çalışma zamanı bağımlılıklarını (`fastapi`, `uvicorn`, `pydantic`) geliştirme/test bağımlılıklarından (`pytest`, `httpx`) ayrı dosyalarda tutmanızın nedeni, üretime dağıtılan bir sunucunun test araçlarını kurması gerekmemesidir.

Terminal

```
$ mkdir sunucu && cd sunucu
$ python3.12 -m venv .venv
$ source .venv/bin/activate
```

```
fastapi~=0.141
uvicorn[standard]~=0.52
pydantic~=2.13
```

```
-r requirements.txt
pytest~=9.1
httpx~=0.28
```

Terminal

```
$ pip install -r requirements.txt -r requirements-dev.txt
```

Beklenen sonuç: Kurulum satırların sonunda başarı mesajı gösterir; sürüm numaralarınız birebir aynı olmayabilir, önemli olan hatasız bitmesidir.

Beklenen Çıktı

```
Installing collected packages: ... fastapi, uvicorn, pydantic, pytest,
httpx, ...
Successfully installed fastapi-0.141.1 pydantic-2.13.5 uvicorn-0.52.4
pytest-9.1.1 httpx-0.28.1 ...
```

macOS / Linux

`python3.12` genelde hazır gelir; gelmiyorsa python.org'dan 3.12 (ya da daha yeni) sürümünü kurun. Aktivasyon her zaman `source .venv/bin/activate` biçimindedir.

Adım 1 (devamı)

Windows

PowerShell'de sanal ortamı `python -m venv .venv` ile oluşturun (sisteminizde `python` zaten 3.12+ ise `python3.12` yerine doğrudan `python` kullanılabilir); etkinleştirmek için `.venv\Scripts\Activate.ps1` çalıştırın. "Çalıştırma ilkesi" hatası alırsanız `Set-ExecutionPolicy -Scope Process RemoteSigned` komutunu bir kez çalıştırın.

İpucu

Sanal ortam etkinken terminal isteminizin başında `(.venv)` yazısı görünür — bugünün geri kalanında bu terminali kapatmayın, yeni bir terminal açtığınızda `source .venv/bin/activate`'i (ya da Windows karşılığını) tekrar çalıştırmanız gerekir.

Adım 2: İlk FastAPI uygulamanızı yazıp `uvicorn` ile çalıştırın

Ne yapacağız? `app/__init__.py` (boş) ve `app/main.py` dosyalarını oluşturacak; `main.py`'de bir `FastAPI` nesnesi ve `"/"` yoluna cevap veren bir kök rota (`kok_rota`) yazacaksınız. Ardından `uvicorn` ile sunucuyu başlatıp `curl` ile bu uç noktaya isteyeceksiniz.

Neden? FastAPI bir ÇERÇEVEdir — hangi yola hangi Python fonksiyonunun cevap vereceğini tanımlar; ama TCP bağlantısını dinleyip HTTP protokolünü konuşan gerçek süreç `uvicorn`dur (bir ASGI sunucusu) — Hafta 1 Ünite 2'nin "web sunucusu" kavramının somut, çalışan bir örneği (bir gişe binası ile gişe memurunun kuralları gibi: bina –uvicorn– kapıyı açar, hangi pencereye yönlendirileceğini kural kitapçığı –FastAPI kodunuz– belirler). `app.main:app` ifadesindeki `app.main` Python modül yoludur, ikinci `app` ise bu modüldeki `FastAPI` nesnesinin adıdır.

```python
app = FastAPI(
    title="KTÜ Etkinlik ve Bilet Sistemi API",
    version="0.1.0",
)
```

```python
@app.get("/")
def kok_rota() -> dict[str, str]:
    return {"mesaj": "KTÜ Etkinlik ve Bilet Sistemi API'sine hoş geldiniz"}
```

Terminal

```
$ uvicorn app.main:app --reload
```

Beklenen sonuç: Terminalde "Uvicorn running on http://127.0.0.1:8000" satırı görünür. Sunucuyu açık bırakıp ayrı bir terminalde `curl -i http://127.0.0.1:8000/` çalıştırdığınızda `200 OK` ile bir hoş geldin JSON'u dönmelidir.

Beklenen Çıktı

```
INFO:     Will watch for changes in these directories: ['.../sunucu']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [.....] using WatchFiles
INFO:     Started server process [.....]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Adım 2 (devamı)

Beklenen Çıktı

```
HTTP/1.1 200 OK
content-type: application/json

{"mesaj":"KTÜ Etkinlik ve Bilet Sistemi API'sine hoş geldiniz"}
```

Ortak Hata / Tuzak

`uvicorn app.main:app` komutunu `sunucu` klasörünün dışından (ör. bir üst dizinden) çalıştırmak "`ModuleNotFoundError: No module named 'app'`" hatası verir — `uvicorn`'u her zaman `app` paketini içeren klasörden (`sunucu/`) çalıştırdığınızı kontrol edin.

Adım 3: Geçici bir yol parametresiyle otomatik dönüşümü/doğrulamayı gözlemleyin

Ne yapacağız? `main.py`'nin en altına, sadece test amacıyla, `etkinlik_id: int` parametresi alan geçici bir `@app.get("/etkinlikler/etkinlik_id")` rotası ekleyeceksiniz; sunucuyu yeniden başlatmadan (`--reload` bunu otomatik yapar) önce geçerli bir tam sayıyla, sonra geçersiz bir metinle bu yola istek göndereceksiniz.

Neden? URL şablonundaki `{etkinlik_id}` yolun değişken bir parçasını yakalar (bir sıra numarasındaki değişen kısım gibi — tabelanın biçimi sabit, sayı her seferinde değişir); fonksiyon parametresine verdiğiniz `int` tip ipucu, FastAPI'nin URL'deki metni otomatik `int`'e çevirmesini sağlar. `"abc"` gibi çevrilemeyen bir değer `404` değil `422 Unprocessable Entity` üretir — yol eşleşiyor (`/etkinlikler/{...}` şablonu var), yalnızca parametrenin tipi uymuyor; `404` yolun kendisi hiç tanımlı değilse dönerdi.

```python
# GECICI TEST -- birazdan gercek rotayla degistirilecek
@app.get("/etkinlikler/{etkinlik_id}")
def gecici_test(etkinlik_id: int) -> dict[str, int]:
    return {"etkinlik_id": etkinlik_id}
```

Terminal

```
$ curl -i http://127.0.0.1:8000/etkinlikler/5
$ curl -i http://127.0.0.1:8000/etkinlikler/abc
```

Beklenen sonuç: İlk istek `200 OK` ile `{"etkinlik_id":5}` döner; ikinci istek `422 Unprocessable Entity` ile `detail` dizisinde `"input should be a valid integer"` benzeri bir mesaj döner.

Beklenen Çıktı

```
HTTP/1.1 422 Unprocessable Entity
content-type: application/json

{"detail":[{"type":"int_parsing","loc":["path","etkinlik_id"],
"msg":"Input should be a valid integer, unable to parse string as an
integer","input":"abc"}]}
```

Beklenen sonuç: Gözlemi bitirince `gecici_test` fonksiyonunu `main.py`'den silin — Bölüm 3'te bu yolu gerçek `etkinlikler` router'ıyla yeniden, tam işlevli biçimde kuracaksınız.

☑ Kontrol Noktası — devam etmeden doğrulayın

Sunucunuz `uvicorn app.main:app --reload` ile açılıyor, kök uç nokta `200 OK` dönüyor ve geçici yol parametresi denemesinde `422`'nin neden `404` değil olduğunu açıklayabiliyorsanız, Ünite 1'in temelini kurmuşsunuz demektir — artık gövde verisini doğrulayan gerçek modelleri yazabilirsiniz.

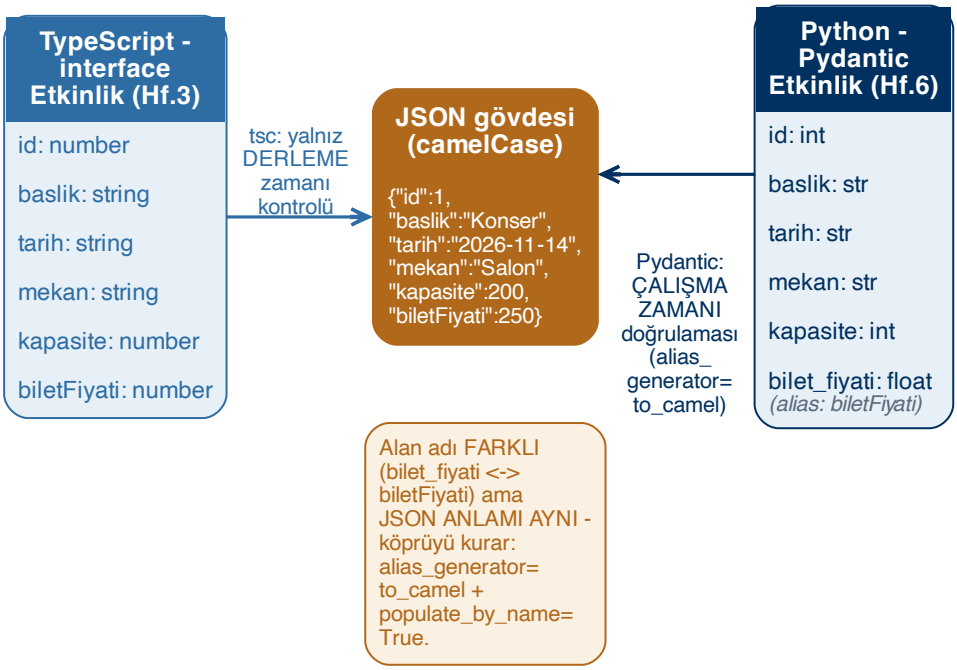
2 Pydantic Modelleri ve Doğrulama

(*bkz. Ders notu Hafta 6, Ünite 2 — Pydantic Modelleri ve Doğrulama.*)

Adım 4: `app/modeller.py` dosyasını yazın

Ne yapacağız? `app/modeller.py` dosyasını oluşturacak; ortak bir `KamelModel` taban sınıfını, `EtkinlikOlustur`/ `Etkinlik` çiftini, `KoltukDurumu` (`Literal`) ile `KoltukOlustur`/`Koltuk`/`KoltukGuncelle`'yi ve `BiletDurumu` ile `BiletOlustur`/`Bilet`'i yazacaksınız.

Neden? Hafta 3'ün `interface Etkinlik`'i yalnızca derleme zamanı bir sözleşmeydi — `tsc` derledikten sonra çalışma zamanında hiçbir kontrol yapılmazdı. Pydantic'in `BaseModel`'i ise çalışma zamanında da gelen veriyi denetler, uymayan veriyi reddeder — bir gişe memurunun formu imzalamadan önce her alanı kontrol etmesi gibi. `EtkinlikOlustur` (kullanıcının gönderdiği, henüz `id`'si olmayan veri) ile `Etkinlik` (sunucunun döndürdüğü, `id` atanmış tam nesne) ayrımı, Hafta 1 Ünite 3'ün "sunum/iş/veri katmanı" ayrımının somut bir yansımasıdır — dışarıdan gelen ile dışarıya giden veri şekli aynı olmak zorunda değildir. `KamelModel`'in `model_config = ConfigDict(alias_generator=to_camel, populate_by_name=True)` satırı, Python tarafında `bilet_fiyati` gibi PEP 8 uyumlu `snake_case` alan adları yazmanıza izin verirken, JSON'a serileştirilirken bunları otomatik olarak `biletFiyati` gibi `camelCase`'e çevirir — Hafta 3'ün TS arayüzleri bu sayede hiç değişmeden bu haftanın JSON çıktısıyla birebir eşleşir.




```python
class KamelModel(BaseModel):
    """snake_case Python alanlarını camelCase JSON'a bağlayan taban sınıf."""
```

Adım 4 (devamı)

```
    model_config = ConfigDict(alias_generator=to_camel, populate_by_name=True)
```

```
class EtkinlikOlustur(KamelModel):
    """İstemcinin gönderdiği, henüz id'si olmayan etkinlik verisi."""

    baslik: str = Field(min_length=1)
    tarih: str
    mekan: str = Field(min_length=1)
    kapasite: int = Field(gt=0)
    bilet_fiyati: float = Field(gt=0)

class Etkinlik(EtkinlikOlustur):
    """Sunucunun döndürdüğü, kimliği atanmış tam etkinlik nesnesi."""

    id: int
```

```
KoltukDurumu = Literal["bos", "secili", "dolu"]

class KoltukOlustur(KamelModel):
    """İstemcinin gönderdiği, henüz id'si/durumu olmayan koltuk verisi."""

    sira: int = Field(gt=0)
    koltuk_no: int = Field(gt=0)

class Koltuk(KoltukOlustur):
    """Sunucunun döndürdüğü, kimliği ve durumu atanmış tam koltuk nesnesi."""

    id: int
    etkinlik_id: int
    durum: KoltukDurumu = "bos"

class KoltukGuncelle(KamelModel):
    """PATCH /koltuklar/{id} gövdesi — yalnızca durum alanı değişir."""

    durum: KoltukDurumu
```

```
BiletDurumu = Literal["olusturuldu", "iptal"]

class BiletOlustur(KamelModel):
    """İstemcinin gönderdiği, henüz id'si/zaman damgası olmayan bilet verisi."""

    koltuk_id: int
    kullanici_adi: str = Field(min_length=1)

class Bilet(BiletOlustur):
    """Sunucunun döndürdüğü, kimliği ve satın alma zamanı atanmış tam bilet."""

    id: int
    satin_alma_zamani: str
    durum: BiletDurumu = "olusturuldu"
```

Beklenen sonuç: Dosyayı kaydettiğinizde editörünüzde hiçbir hata görünmez — bu modelleri henüz hiçbir rota kullanmıyor, sunucuda hiçbir değişiklik görmeyeceksiniz. Aşağıdaki iki komutla köprüyü ve doğrulamayı, sunucuyu hiç çalıştırmadan, doğrudan Python içinde sınayabilirsiniz.

Adım 4 (devamı)

Terminal

```
$ python -c "from app.modeller import EtkinlikOlustur; e =
↪  EtkinlikOlustur(baslik='Konser', tarih='2026-11-14', mekan='Salon',
↪  kapasite=100, bilet_fiyati=250); print(e.model_dump(by_alias=True))"
```

Beklenen Çıktı

```
{'baslik': 'Konser', 'tarih': '2026-11-14', 'mekan': 'Salon',
'kapasite': 100, 'biletFiyati': 250.0}
```

Terminal

```
$ python -c "from app.modeller import EtkinlikOlustur;
↪  EtkinlikOlustur(baslik='Konser', tarih='2026-11-14', mekan='Salon',
↪  kapasite=-3, bilet_fiyati=250)"
```

Beklenen sonuç: İkinci komut bilerek bir hata üretir — `kapasite=-3` `Field(gt=0)` kısıtını ihlal eder, Pydantic bir `ValidationError` fırlatır ve nesne hiç oluşturulmaz.

Beklenen Çıktı

```
pydantic_core._pydantic_core.ValidationError: 1 validation error for
EtkinlikOlustur
kapasite
  Input should be greater than 0 [type=greater_than, input_value=-3,
  input_type=int]
```

Ortak Hata / Tuzak

`model_dump()`'ı `by_alias=True` olmadan çağırmak `snake_case` alan adlarını (`bilet_fiyati`) döndürür — FastAPI'nin bir rota içinde otomatik ürettiği gerçek JSON cevabı ise varsayılan olarak `camelCase` takma adı kullanır; bu ikisini karıştırmayın, `by_alias=True` yalnızca bu elle yaptığınız Python testinde gereklidir.

Kontrol Noktası — devam etmeden doğrulayın

`modeller.py` dosyanız hatasız duruyorsa, `by_alias=True` ile `biletFiyati`'yi gördüyseniz ve negatif kapasitenin bir `ValidationError` fırlattığını gözlemlediyseniz, Ünite 2'nin temelini kurmuşsunuz demektir — artık bu modelleri gerçek uç noktalara bağlayabilirsiniz.

3 CRUD, Durum Kodları, Hata Cevapları

(bkz. Ders notu Hafta 6, Ünite 3 — CRUD, Durum Kodları, Hata Cevapları.)

Adım 5: `app/depo.py` dosyasını yazın

Ne yapacağız? `app/depo.py` dosyasını oluşturacak; üç `dict[int, ...]` (etkinlikler/koltuklar/biletler) ve üç sayaç tutan `Depo` sınıfını, her varlık için ekle/getir/listele/ güncelle/sil metotlarını ve modül düzeyinde tek bir `depo = Depo()` nesnesini yazacaksınız.

Adım 5 (devamı)

Neden? `dict[int, X]`, YG I/II'de yazdığınız `std::unordered_map` benzeri bir anahtar-değer deposudur — bir toplantı odasındaki beyaz tahta gibi düşünün: üzerine yazdığınız her şey oda açıkken görünür ve paylaşılır, ama oda temizlendiğinde (sunucu yeniden başladığında) tahtadaki her şey silinir — kalıcı bir defter (veritabanı) değildir. Modül düzeyinde tek bir `depo` nesnesi tanımlamanız önemlidir: birazdan yazacağınız üç router (Etkinlikler/Koltuklar/Biletler) aynı nesneyi paylaşacak, bu sayede bir `POST /etkinlikler`'de eklenen etkinlik hemen ardından bir `GET /etkinlikler`'de görünür.

```python
class Depo:
    """Etkinlik/Koltuk/Bilet kayıtlarını `dict[int, ...]` ile bellekte tutar."""

    def __init__(self) -> None:
        self._etkinlikler: dict[int, Etkinlik] = {}
        self._koltuklar: dict[int, Koltuk] = {}
        self._biletler: dict[int, Bilet] = {}
        self._sonraki_etkinlik_id = 1
        self._sonraki_koltuk_id = 1
        self._sonraki_bilet_id = 1

    # -- Etkinlik ------------------------------------------------------
    def etkinlik_ekle(self, veri: EtkinlikOlustur) -> Etkinlik:
        etkinlik = Etkinlik(id=self._sonraki_etkinlik_id, **veri.model_dump())
        self._etkinlikler[etkinlik.id] = etkinlik
        self._sonraki_etkinlik_id += 1
        return etkinlik

    def etkinlik_getir(self, etkinlik_id: int) -> Etkinlik | None:
        return self._etkinlikler.get(etkinlik_id)

    def etkinlik_listele(self, mekan: str | None = None) -> list[Etkinlik]:
        etkinlikler = list(self._etkinlikler.values())
        if mekan is not None:
            etkinlikler = [e for e in etkinlikler if e.mekan == mekan]
        return etkinlikler

    def etkinlik_guncelle(
        self, etkinlik_id: int, veri: EtkinlikOlustur
    ) -> Etkinlik | None:
        if etkinlik_id not in self._etkinlikler:
            return None
        etkinlik = Etkinlik(id=etkinlik_id, **veri.model_dump())
        self._etkinlikler[etkinlik_id] = etkinlik
        return etkinlik

    def etkinlik_sil(self, etkinlik_id: int) -> bool:
        return self._etkinlikler.pop(etkinlik_id, None) is not None

    # -- Koltuk --------------------------------------------------------
    def koltuk_ekle(self, etkinlik_id: int, veri: KoltukOlustur) -> Koltuk:
        koltuk = Koltuk(
            id=self._sonraki_koltuk_id,
            etkinlik_id=etkinlik_id,
            durum="bos",
            **veri.model_dump(),
        )
        self._koltuklar[koltuk.id] = koltuk
```

Adım 5 (devamı)

```
        self._sonraki_koltuk_id += 1
        return koltuk

    def koltuk_getir(self, koltuk_id: int) -> Koltuk | None:
        return self._koltuklar.get(koltuk_id)

    def koltuk_listele(
        self, etkinlik_id: int, durum: KoltukDurumu | None = None
    ) -> list[Koltuk]:
        koltuklar = [
            k for k in self._koltuklar.values() if k.etkinlik_id == etkinlik_id
        ]
        if durum is not None:
            koltuklar = [k for k in koltuklar if k.durum == durum]
        return koltuklar

    def koltuk_guncelle(self, koltuk_id: int, durum: KoltukDurumu) -> Koltuk | None:
        koltuk = self._koltuklar.get(koltuk_id)
        if koltuk is None:
            return None
        guncel = koltuk.model_copy(update={"durum": durum})
        self._koltuklar[koltuk_id] = guncel
        return guncel

    def koltuk_sil(self, koltuk_id: int) -> bool:
        return self._koltuklar.pop(koltuk_id, None) is not None

    # -- Bilet ------------------------------------------------------------
    def bilet_ekle(self, veri: BiletOlustur) -> Bilet:
        bilet = Bilet(
            id=self._sonraki_bilet_id,
            satin_alma_zamani=datetime.now().isoformat(),
            durum="olusturuldu",
            **veri.model_dump(),
        )
        self._biletler[bilet.id] = bilet
        self._sonraki_bilet_id += 1
        return bilet

    def bilet_getir(self, bilet_id: int) -> Bilet | None:
        return self._biletler.get(bilet_id)

    def bilet_listele(self, kullanici_adi: str | None = None) -> list[Bilet]:
        biletler = list(self._biletler.values())
        if kullanici_adi is not None:
            biletler = [b for b in biletler if b.kullanici_adi == kullanici_adi]
        return biletler

    def bilet_iptal_et(self, bilet_id: int) -> Bilet | None:
        """Bileti iptal eder (depodan SİLMEZ) ve ilgili koltuğu boşaltır."""
        bilet = self._biletler.get(bilet_id)
        if bilet is None:
            return None
        guncel_bilet = bilet.model_copy(update={"durum": "iptal"})
        self._biletler[bilet_id] = guncel_bilet
        koltuk = self._koltuklar.get(bilet.koltuk_id)
```

Adım 5 (devamı)

```
        if koltuk is not None:
            self._koltuklar[koltuk.id] = koltuk.model_copy(update={"durum": "bos"})
        return guncel_bilet
```

```
# Tüm rotalar (Etkinlikler/Koltuklar/Biletler) AYNI depoyu kullanır; bu
# sayede bir POST /etkinlikler'de eklenen etkinlik HEMEN ardından bir
# GET /etkinlikler'de görünür.
depo = Depo()
```

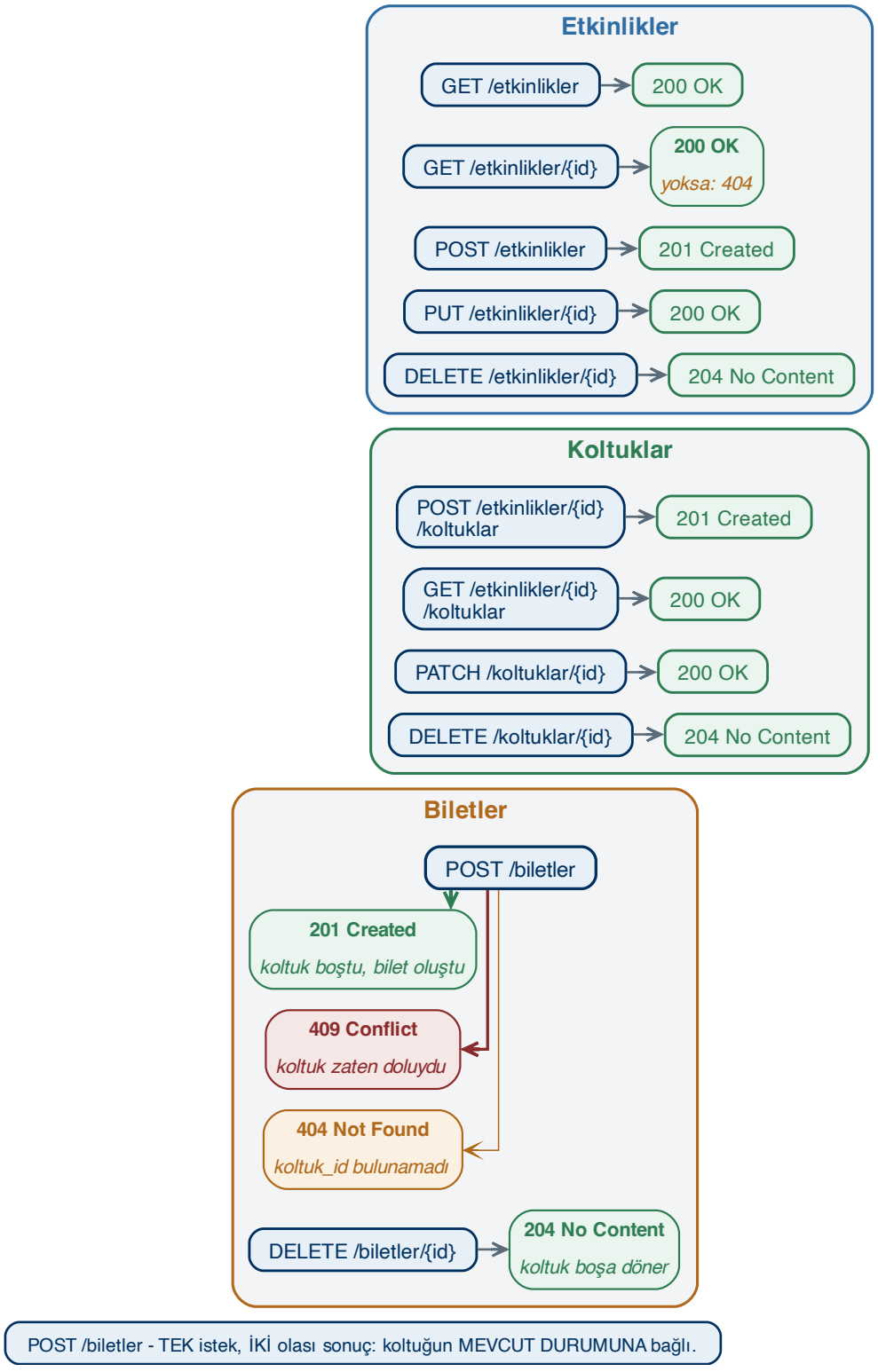
Beklenen sonuç: Dosya hatasız kaydedilir; `depo.py` henüz hiçbir rota tarafından kullanılmıyor, sunucuda görünür bir değişiklik yoktur.

Adım 6: Etkinlik rotalarını yazıp gerçek bir CRUD döngüsünü deneyin

Ne yapacağız? `app/bagimliliklar.py` dosyasını oluşturup `depo`'yu döndüren `get_depo()` fonksiyonunu yazacaksınız. Ardından `app/rotalar/__init__.py` (boş) ve `app/rotalar/etkinlikler.py` dosyalarını oluşturup `APIRouter(prefix="/etkinlikler", ...)` ile liste (`GET`, `mekan` sorgu parametreli), detay (`GET /{etkinlik_id}`), oluşturma (`POST`, 201), güncelleme (`PUT`, 200) ve silme (`DELETE`, 204) rotalarını yazacaksınız. Son olarak `main.py`'deki geçici test rotasını kaldırıp dosyanın en üstüne `from app.rotalar import etkinlikler` satırını, en altına ise `app.include_router(etkinlikler.router)` satırını ekleyeceksiniz.

Neden? `get_depo()` şimdilik yalnızca "`depo`'yu döndüren küçük bir yardımcı fonksiyon" olarak düşünün — resmî adını (*bağımlılık enjeksiyonu*) Bölüm 4'te göreceksiniz; şimdilik her rota fonksiyonunun parametre listesine `depo: Depo = Depends(get_depo)` yazdığınızı ve bunun size paylaşılan `depo` nesnesini verdiğini bilmeniz yeterlidir. Tüm rotaları tek bir `main.py` yerine ayrı bir `APIRouter` dosyasında toplamanız, YG II'de büyük bir `.cpp` dosyasını sorumluluğa göre birden fazla dosyaya bölme disiplininin FastAPI karşılığıdır — `etkinlikler.py` yalnızca Etkinlik'le ilgilenir. `POST 201 Created` (yeni kayıt), `GET 200 OK` (okuma), `PUT 200 OK` (tüm alanları değiştiren tam güncelleme), `DELETE 204 No Content` (gövdesiz, kayıt tamamen kaldırılır) — Hafta 1 Ünite 4'ün durum kodu ailelerinin somut kullanımıdır. `HTTPException(status_code=404, ...)`, istenen `id` depoda yoksa fırlatılan ve FastAPI'nin otomatik bir JSON hata gövdesine çevirdiği istisnadır.

Adım 6 (devamı)




```python
def get_depo() -> Depo:
    """Rota fonksiyonlarına PAYLAŞILAN tek Depo nesnesini enjekte eder.

    Hf.8'de bu fonksiyonun yerini bir veritabanı oturumu enjekte eden
    get_db_session() alacak; rota fonksiyonlarının bağımlılık ALMA
```

Adım 6 (devamı)

```
    biçimi değişmeyecek, yalnızca bağımlılığın NE OLDUĞU değişecek.
    """
    return depo
```

```
router = APIRouter(prefix="/etkinlikler", tags=["etkinlikler"])
```

```
@router.get("", response_model=list[Etkinlik])
def etkinlik_liste(
    mekan: str | None = None, depo: Depo = Depends(get_depo)
) -> list[Etkinlik]:
    return depo.etkinlik_listele(mekan=mekan)
```

```
@router.get("/{etkinlik_id}", response_model=Etkinlik)
def etkinlik_detay(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> Etkinlik:
    etkinlik = depo.etkinlik_getir(etkinlik_id)
    if etkinlik is None:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    return etkinlik
```

```
@router.post("", response_model=Etkinlik, status_code=201)
def etkinlik_olustur(
    veri: EtkinlikOlustur, depo: Depo = Depends(get_depo)
) -> Etkinlik:
    return depo.etkinlik_ekle(veri)
```

```
@router.put("/{etkinlik_id}", response_model=Etkinlik)
def etkinlik_guncelle(
    etkinlik_id: int, veri: EtkinlikOlustur, depo: Depo = Depends(get_depo)
) -> Etkinlik:
    etkinlik = depo.etkinlik_guncelle(etkinlik_id, veri)
    if etkinlik is None:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    return etkinlik
```

```
@router.delete("/{etkinlik_id}", status_code=204)
def etkinlik_sil(etkinlik_id: int, depo: Depo = Depends(get_depo)) -> None:
    silindi = depo.etkinlik_sil(etkinlik_id)
    if not silindi:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
```

Terminal

```
$ curl -i -X POST http://127.0.0.1:8000/etkinlikler \
  -H "Content-Type: application/json" \
  -d '{"baslik": "Konser", "tarih": "2026-11-14", "mekan": "KTÜ Kültür Merkezi",
  ↪  "kapasite": 100, "biletFiyati": 250}'
```

Beklenen sonuç: `201 Created` döner; cevap gövdesinde oluşturulan nesnenin tamamı, `id` dahil, camelCase alanlarla görünür.

Beklenen Çıktı

```
HTTP/1.1 201 Created
content-type: application/json

{"baslik":"Konser","tarih":"2026-11-14","mekan":"KTÜ Kültür
Merkezi","kapasite":100,"biletFiyati":250.0,"id":1}
```

Adım 6 (devamı)

Terminal

```
$ curl -i http://127.0.0.1:8000/etkinlikler
$ curl -i -X PUT http://127.0.0.1:8000/etkinlikler/1 \
  -H "Content-Type: application/json" \
  -d '{"baslik": "Konser (Güncel)", "tarih": "2026-11-15", "mekan": "KTÜ Kültür
  ↪  Merkezi", "kapasite": 120, "biletFiyati": 300}'
```

Beklenen sonuç: İlk istek yeni eklediğiniz etkinliği içeren bir liste (`200`) döner; ikinci istek `baslik`/`tarih`/ `kapasite`/`biletFiyati`'nin tamamen güncellendiği `200` cevabı döner.

Beklenen Çıktı

```
HTTP/1.1 200 OK
content-type: application/json

{"baslik":"Konser (Güncel)","tarih":"2026-11-15","mekan":"KTÜ Kültür
Merkezi","kapasite":120,"biletFiyati":300.0,"id":1}
```

Dene-Çalıştır

`curl -i http://127.0.0.1:8000/etkinlikler/999` ile olmayan bir `id` isteyin — `404 Not Found` ile `{"detail": "Etkinlik bulunamadı"}` döndüğünü gözlemleyin; ardından `curl -i -X DELETE http://127.0.0.1:8000/etkinlikler/1` ile etkinliği silin (`204`, gövdesiz) ve aynı `id` için tekrar `GET` yaparak bu kez `404` aldığınızı doğrulayın — silinen bir kayıt artık hiçbir yerde yoktur.

Adım 7: Koltuk ve bilet rotalarını yazıp koltuk rezervasyonu iş kuralını deneyin

Ne yapacağız? `app/rotalar/koltuklar.py` dosyasını oluşturup (`prefix` olmadan bir `APIRouter`) koltuk oluşturma (`POST /etkinlikler/{etkinlik_id}/koltuklar`), listeleme (`GET`, `durum` sorgu parametreli), detay, durum güncelleme (`PATCH`, `KoltukGuncelle` gövdesi) ve silme rotalarını yazacaksınız. Ardından `app/rotalar/biletler.py` dosyasını oluşturup bilet oluşturmayı (`POST`, koltuk rezervasyonu iş kuralıyla), listeleme, detay ve iptal (`DELETE`) rotalarını yazacaksınız. Son olarak `main.py`'nin en üstündeki içe aktarma satırını `from app.rotalar import biletler, etkinlikler, koltuklar` olacak biçimde güncelleyip, en alttaki iki `app.include_router(...)` satırını ekleyeceksiniz.

Neden? `Koltuk` için `PUT` değil `PATCH` kullanmanızın nedeni, yalnızca `durum` alanını kısmi güncellemedir — "tümünü değiştir" (`PUT`) ile "yalnızca belirtileni değiştir" (`PATCH`) arasındaki fark budur. `POST /biletler` çağrıldığında önce `koltuk_id`'nin depoda var olup olmadığı kontrol edilir (yoksa `404`); varsa koltuğun `durum`'u denetlenir — `'dolu'` ise istek `409 Conflict` ile reddedilir (aynı numaralı otobüs koltuğunu iki kişinin ayrı gişelerden almaya çalışması gibi: ikinci istek format olarak tamamen doğru olabilir, ama gişe memuru "bu koltuk zaten satıldı" der — sorun formatta değil, mevcut durumdadır). `'bos'` ise bilet oluşturulur, koltuğun `durum`'u `'dolu'`ya çevrilir ve `201` dönülür. `DELETE /biletler/{id}` bileti depodan silmez — `durum`'unu `'iptal'`e çevirir ve koltuğu `'bos'`a döndürür (denetim için iptal edilen biletlerin saklanması fikri).

```
1 router = APIRouter(tags=["koltuklar"])
```

```
1 @router.post(
```

Adım 7 (devamı)

```
    "/etkinlikler/{etkinlik_id}/koltuklar", response_model=Koltuk, status_code=201
)
def koltuk_olustur(
    etkinlik_id: int, veri: KoltukOlustur, depo: Depo = Depends(get_depo)
) -> Koltuk:
    etkinlik = depo.etkinlik_getir(etkinlik_id)
    if etkinlik is None:
        raise HTTPException(status_code=404, detail="Etkinlik bulunamadı")
    return depo.koltuk_ekle(etkinlik_id, veri)
```

```
@router.patch("/koltuklar/{koltuk_id}", response_model=Koltuk)
def koltuk_durum_guncelle(
    koltuk_id: int, veri: KoltukGuncelle, depo: Depo = Depends(get_depo)
) -> Koltuk:
    koltuk = depo.koltuk_guncelle(koltuk_id, veri.durum)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    return koltuk
```

```
router = APIRouter(prefix="/biletler", tags=["biletler"])
```

```
@router.post("", response_model=Bilet, status_code=201)
def bilet_olustur(veri: BiletOlustur, depo: Depo = Depends(get_depo)) -> Bilet:
    koltuk = depo.koltuk_getir(veri.koltuk_id)
    if koltuk is None:
        raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
    if koltuk.durum == "dolu":
        raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
    bilet = depo.bilet_ekle(veri)
    depo.koltuk_guncelle(koltuk.id, "dolu")
    return bilet
```

```
@router.delete("/{bilet_id}", status_code=204)
def bilet_iptal(bilet_id: int, depo: Depo = Depends(get_depo)) -> None:
    bilet = depo.bilet_iptal_et(bilet_id)
    if bilet is None:
        raise HTTPException(status_code=404, detail="Bilet bulunamadı")
```

```
app.include_router(etkinlikler.router)
app.include_router(koltuklar.router)
app.include_router(biletler.router)
```

Aşağıdaki `istekler.sh` betiği (`sunucu` çalışırken `bash istekler.sh` ile çalıştırılır) bu turun tamamını otomatikleştirir:

```
curl -i -X POST "$BASE_URL/etkinlikler/1/koltuklar" \
  -H "Content-Type: application/json" \
  -d '{"sira": 1, "koltukNo": 5}'
```

```
curl -i -X POST "$BASE_URL/biletler" \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Ayşe Yılmaz"}'
```

```
# Aynı koltuk için İKİNCİ bir bilet denemesi — 409 Conflict bekleyin.
curl -i -X POST "$BASE_URL/biletler" \
  -H "Content-Type: application/json" \
  -d '{"koltukId": 1, "kullaniciAdi": "Mehmet Demir"}'
```

Adım 7 (devamı)

Beklenen sonuç: Koltuk oluşturma `201` ile `durum:"bos"` döner; ilk bilet isteği `201` ile döner ve koltuğun durumu `'dolu'`ya geçer; aynı koltuğa ikinci bilet isteği `409 Conflict` döner.

Beklenen Çıktı

```
HTTP/1.1 201 Created
{"sira":1,"koltukNo":5,"id":1,"etkinlikId":1,"durum":"bos"}

HTTP/1.1 201 Created
{"koltukId":1,"kullaniciAdi":"Ayşe Yılmaz","id":1,
"satinAlmaZamani":"2026-09-07T15:21:35.096828","durum":"olusturuldu"}

HTTP/1.1 409 Conflict
{"detail":"Bu koltuk zaten satılmış"}
```

Ortak Hata / Tuzak

Koltuk rezervasyonu iş kuralında önce `koltuk.durum == 'dolu'` kontrolünü yapmadan doğrudan `depo.bilet_ekle(veri)` çağırmak, aynı koltuk için sınırsız sayıda bilet oluşturulmasına izin verir — kontrolün `depo.bilet_ekle`'den önce, koltuğu getirdikten hemen sonra yapıldığından emin olun.

Kontrol Noktası — devam etmeden doğrulayın

`curl` ile bir etkinlik oluşturup listeleyebiliyor, güncelleyip silebiliyorsanız ve bir koltuğa ikinci bilet denemenizin `409` ile reddedildiğini gördüyseniz, Ünite 3'ün CRUD/durum kodu/iş kuralı üçlüsünü kurmuşsunuz demektir — artık bu API'yi tek bir tarayıcı sayfasında gezilebilir hâle getirip test edebilirsiniz.

4 OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu Girişi

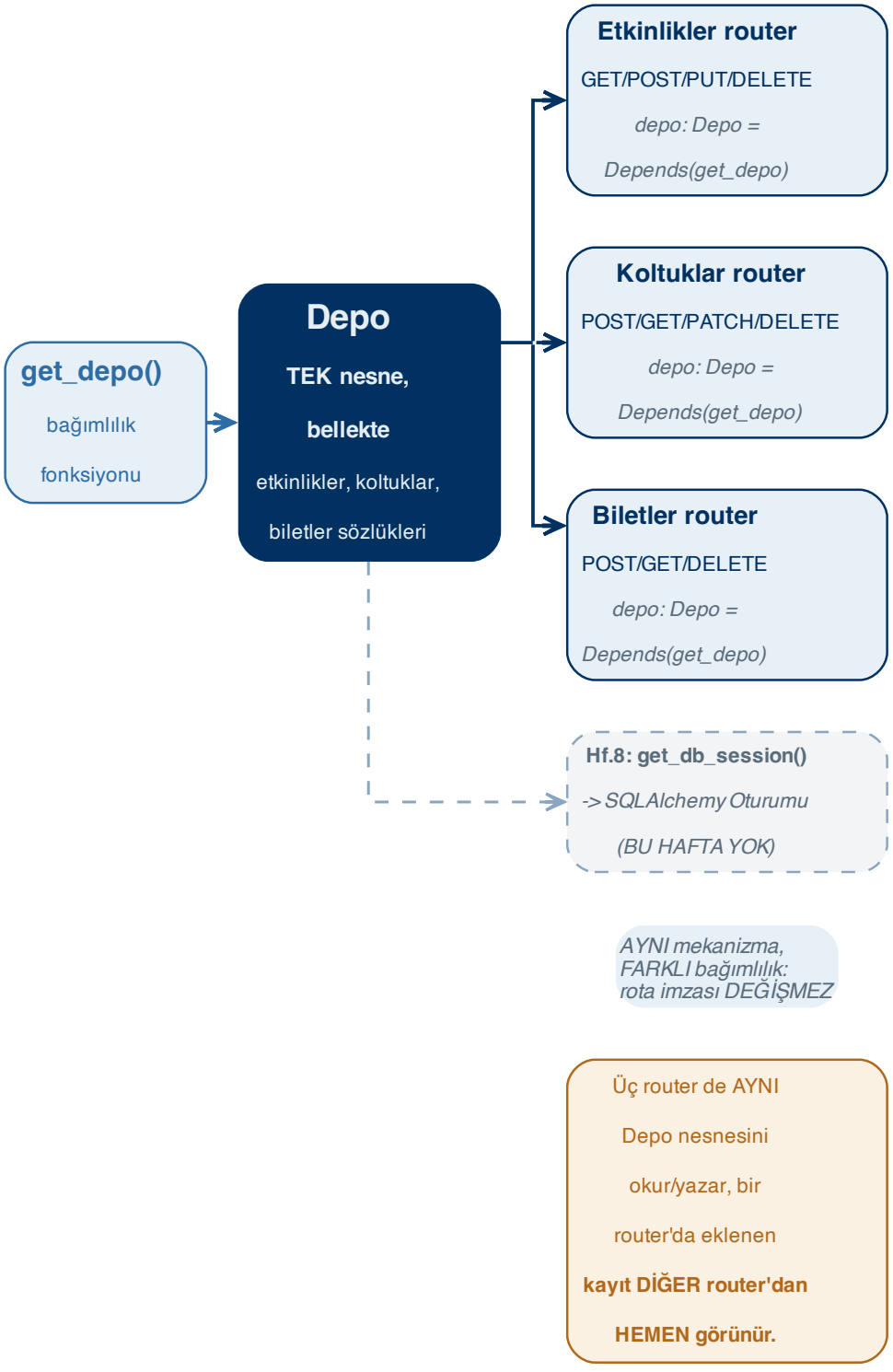
(bkz. Ders notu Hafta 6, Ünite 4 — OpenAPI Belgeleri ve Bağımlılık Enjeksiyonu Girişi.)

Adım 8: `/docs` ve `/openapi.json`'ı inceleyip Hafta 3'ün TS tipleriyle karşılaştırın

Ne yapacağız? Sunucu çalışırken tarayıcıda `http://127.0.0.1:8000/docs` adresini açacak, `POST /biletler` uç noktasını genişletip "Try it out" ile örnek bir istek göndereceksiniz. Ardından `http://127.0.0.1:8000/openapi.json`'ı açıp `Bilet` şemasının alan adlarını Hafta 3'ün `interface Bilet`'iyle karşılaştıracaksınız.

Neden? `/docs` (Swagger UI), yazdığınız her rotadan otomatik üretilen, tarayıcıda açılıp her uç noktayı deneyebileceğiniz interaktif bir belge sayfasıdır — bir restoranın, mutfak tarafından otomatik güncellenen menüsü gibi: siz yeni bir rota eklediğinizde menü kendiliğinden güncellenir, elle bir belge yazmanıza gerek kalmaz. Bu belge, Bölüm 3'te yazdığınız `Depends(get_depo)` kullanımının da bir sonucudur: `POST /etkinlikler`'de eklediğiniz bir etkinliğin hemen ardından `GET /etkinlikler`'de görünmesi — üç router'ın da (Etkinlikler/Koltuklar/Biletler) aynı `depo` nesnesini kullanması sayesindedir. `Depends(get_depo)` Yerine her rota fonksiyonunun içinde doğrudan `Depo()` çağırıp yeni bir nesne oluştursaydınız, her istek kendi taze, boş deposunu görürdü — eklediğiniz hiçbir kayıt bir sonraki istekte görünmezdi.

Adım 8 (devamı)




```
>_ Terminal
$ curl -s -o /dev/null -w "docs: %{http_code}\n" http://127.0.0.1:8000/docs
```

Beklenen sonuç: Tarayıcıda `/docs` sayfasında tüm rotalarınız (etkinlik/ koltuk/bilet), olası durum kodları (201/404/409) ve gövde şemaları görünür; `/openapi.json`'daki `Bilet` şemasının `properties` listesi Hafta 3'ün `interface Bilet { id, koltukId, kullaniciAdi, satinAlmaZamani, durum }` alanlarıyla birebir (camelCase) örtüşür.

Adım 8 (devamı)

Beklenen Çıktı

```
docs: 200
['koltukId', 'kullaniciAdi', 'id', 'satinAlmaZamani', 'durum']
```

İpucu

CORS'a (Cross-Origin Resource Sharing) hiç değinmedik — tarayıcıdan bu API'ye `fetch` ile bağlanmaya çalıştığınızda bir engelle karşılaşacaksınız; çözümü Hafta 7'nin konusudur, bugün kod örneği içermez.

Adım 9: İlk API testlerinizi `TestClient` ile yazın

Ne yapacağız? `tests/conftest.py` dosyasını oluşturup her teste temiz, boş bir `Depo` veren `client` fixture'ını yazacaksınız. Ardından `tests/test_etkinlikler.py` dosyasını oluşturup oluşturma+listeleme, `mekan` filtresi, `404`, `PUT` ile güncelleme+silme ve negatif kapasiteyle `422` testlerini yazacaksınız.

Neden? `TestClient`, gerçek bir sunucu ayağa kaldırmadan, doğrudan Python içinde HTTP benzeri istek gönderen bir test aracıdır — YG II'nin `assert` disiplini burada `assert response.status_code == 201` biçiminde aynen devam eder. Testte `app.dependency_overrides[get_depo] = lambda: test_depo` satırı, `Depends`'in gerçek değerini gösterir: eğer rota fonksiyonları depoyu kendileri oluştursaydı, testler arasında veri sızıntısı (bir testin oluşturduğu kaydı bir sonraki testin görmesi) engellenemezdi; `Depends(get_depo)` sayesinde her test kendi temiz deposuyla başlar.

```python
@pytest.fixture
def client():
    test_depo = Depo()
    app.dependency_overrides[get_depo] = lambda: test_depo
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()
```

```python
def test_etkinlik_olustur_listele(client):
    yeni = {
        "baslik": "Konser",
        "tarih": "2026-11-14",
        "mekan": "KTÜ Kültür Merkezi",
        "kapasite": 100,
        "biletFiyati": 250,
    }
    cevap = client.post("/etkinlikler", json=yeni)
    assert cevap.status_code == 201
    olusturulan = cevap.json()
    assert olusturulan["id"] == 1
    assert olusturulan["biletFiyati"] == 250

    liste_cevabi = client.get("/etkinlikler")
    assert liste_cevabi.status_code == 200
    assert len(liste_cevabi.json()) == 1
```

```python
def test_etkinlik_dogrulama_422(client):
    cevap = client.post(
```

Adım 9 (devamı)

```
        "/etkinlikler",
        json={
            "baslik": "Konser",
            "tarih": "2026-11-14",
            "mekan": "Salon A",
            "kapasite": -3,
            "biletFiyati": 250,
        },
    )
    assert cevap.status_code == 422
    detay = cevap.json()["detail"]
    assert detay[0]["loc"] == ["body", "kapasite"]
```

Terminal

```
$ python -m pytest tests/test_etkinlikler.py -q
```

Beklenen sonuç: Beş test de hatasız geçer.

Beklenen Çıktı

```
.....                                                    [100%]
5 passed, 2 warnings in 0.02s
```

Dikkat

İki uyarı (`DeprecationWarning`) `starlette/httpx` kütüphanelerinin kendi iç uyarılarıdır — sizin kodunuzdan kaynaklanmaz, testleri etkilemez; brifte sabitlenen sürümlerle (bu haftanın `httpx ~0.28`'i) ilgilidir.

Adım 10: Bilet testlerini yazıp `python -m pytest -q` kapı komutunu çalıştırın

Ne yapacağız? `tests/test_biletler.py` dosyasını oluşturup başarılı rezervasyon, çift rezervasyon (`409`), olmayan koltuğa bilet (`404`) ve bilet iptali testlerini yazacaksınız. Ardından tüm test paketini çalıştıracaksınız.

Neden? Bu dört test, Bölüm 3'te `curl` ile elle denediğiniz koltuk rezervasyonu iş kuralını kalıcı, tekrarlanabilir bir biçimde sabitler — kodunuzu değiştirdiğinizde bu testleri tekrar çalıştırarak `409` davranışının bozulmadığını saniyeler içinde doğrulayabilirsiniz. Bu, dönem boyunca (Hafta 8'in transaction testlerinde, Hafta 14'ün CI adımında) aynı `pytest -q` komutuyla tekrar karşınıza çıkacak bir alışkanlıktır.

```
def test_bilet_basarili_rezervasyon(client):
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)

    cevap = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert cevap.status_code == 201
    assert cevap.json()["durum"] == "olusturuldu"

    guncel_koltuk = client.get(f"/koltuklar/{koltuk['id']}")
    assert guncel_koltuk.json()["durum"] == "dolu"
```

Adım 10 (devamı)

```python
def test_bilet_coklu_rezervasyon_409(client):
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)

    ilk = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert ilk.status_code == 201

    ikinci = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Mehmet Demir"}
    )
    assert ikinci.status_code == 409
    assert "satılmış" in ikinci.json()["detail"]
```

```python
def test_bilet_iptal(client):
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)
    bilet = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    ).json()

    iptal = client.delete(f"/biletler/{bilet['id']}")
    assert iptal.status_code == 204

    guncel_koltuk = client.get(f"/koltuklar/{koltuk['id']}")
    assert guncel_koltuk.json()["durum"] == "bos"
```

Terminal

```
$ python -m pytest -q
```

Beklenen sonuç: Tüm testler (5 etkinlik + 4 bilet = 9) hatasız geçer — bu, bugünün kod örneklerinin "kapı" komutudur.

Beklenen Çıktı

```
.........                                                [100%]
9 passed, 2 warnings in 0.03s
```

Kontrol Noktası — devam etmeden doğrulayın

`/docs` sayfasında tüm rotalarınızı gördüyseniz, `openapi.json`'daki `Bilet` şemasını Hafta 3'ün TS tipiyle karşılaştırdıysanız ve `python -m pytest -q` çıktısında `9 passed` gördüyseniz, bu haftanın dört ünitesini de uygulamışsınız demektir — Etkinlik/Bilet Sisteminin ilk sunucu katmanı tamamlanmıştır.

Adım 11: Kendiniz ekleyin: koltuk durum güncellemesi için bir test yazın

Ne yapacağız? `tests/test_koltuklar.py` dosyasını oluşturup, bir etkinlik ve bir koltuk oluşturduktan sonra `PATCH /koltuklar/{koltuk_id}` ile bu koltuğun `durum`'unu **bilet oluşturmadan, doğrudan** `'dolu'`ya çeviren bir test yazacaksınız.

Neden? Şimdiye kadarki testler koltuğun `durum`'unun yalnızca bir bilet rezervasyonu sonucunda değiştiğini doğruluyordu; bu görev, `PATCH /koltuklar/{id}` uç noktasının —

Adım 11 (devamı)

`KoltukGuncelle` gövdesiyle — tek başına da doğru çalıştığını sizin yazdığınız bir testle kanıtlamanızı sağlar. `tests/test_biletler.py`'deki `_etkinlik_ve_koltuk_olustur` yardımcı fonksiyonunu örnek alıp aynı kurulum deseni (önce etkinlik, sonra koltuk oluşturma) izleyin.

Beklenen sonuç: Yeni testi ekledikten sonra `python -m pytest -q` çalıştırdığınızda toplam test sayısı **10**'a çıkar, tümü hatasız geçer.

Beklenen Çıktı

```
..........                                                [100%]
10 passed, 2 warnings in 0.04s
```

İpucu

`client.patch(f"/koltuklar/{koltuk_id}", json={"durum": "dolu"})` çağrısının dönüş değerinin `status_code`'unu (`200` bekleyin) ve `json()["durum"]`'unu (`"dolu"` bekleyin) `assert` ile denetleyin — `tests/ test_etkinlikler.py`'deki `test_etkinlik_guncelle_sil` testinin yapısı size bir şablon sağlar.

Sorun Giderme

Belirti	Olası neden	Çözüm
`uvicorn: command not found` ya da `ModuleNotFoundError: No module named 'fastapi'`	Sanal ortam etkinleştirilmemiş ya da bağımlılıklar bu ortama kurulmamış	`source .venv/bin/activate` (Windows: `.venv\Scripts\Activate.ps1`) çalıştırıp `pip install -r requirements.txt -r requirements-dev.txt`'yi tekrarlayın
Kök uç nokta yerine `{"detail":"Not Found"}` ile `404` dönüyor	Yola yanlışlıkla sonda bir `/` eksik/fazla eklenmiş ya da `uvicorn` farklı bir port/modülle başlatılmış	URL'yi `http://127.0.0.1:8000/` olarak birebir kontrol edin; `uvicorn app.main:app` komutunun `sunucu` klasöründen çalıştığını doğrulayın
`POST /biletler` her zaman `201` dönüyor, `409` hiç görünmüyor	Koltuğun `durum`'unu kontrol eden `if koltuk.durum == "dolu"` satırı eksik ya da `depo.koltuk_guncelle` çağrısı unutulmuş	`bilet_olustur` fonksiyonunda kontrolün `depo.bilet_ekle`'den önce yapıldığını ve bilet eklendikten sonra koltuğun `'dolu'`ya güncellendiğini doğrulayın
JSON gövdesinde `snake_case` alan adları (`bilet_fiyati`) görünüyor, camelCase değil	Model `KamelModel`'den değil doğrudan `BaseModel`'den türetilmiş	Tüm varlık modellerinin (`Etkinlik`, `Koltuk`, `Bilet` ve `... Olustur` varyantları) `KamelModel`'den türediğini kontrol edin
`python -m pytest -q` "`fixture 'client' not found`" hatası veriyor	`tests/conftest.py` dosyası yok ya da `client` fixture'ı yanlış isimle tanımlanmış	`conftest.py`'nin `tests/` klasöründe olduğunu ve fixture fonksiyon adının birebir `client` olduğunu doğrulayın
Sunucuyu kapattıktan sonra `uvicorn` arka planda çalışmaya devam ediyor, port yeniden başlatılamıyor	Terminal penceresi kapatılmadan önce `Ctrl+C` ile sunucu durdurulmamış	`uvicorn`'un çalıştığı terminalde `Ctrl+C` basın; port hâlâ meşgulse `lsof -i :8000` (macOS/Linux) ile süreci bulup sonlandırın

5 • İzleme — Aktarım — Başka Nerede?

Bugün Etkinlik/Koltuk/Bilet için venv+pip ile kurduğunuz FastAPI projesini, tip ipuçlarından otomatik doğrulama üreten Pydantic modellerini, doğru durum koduyla cevap veren CRUD rotalarını ve `Depends`'le paylaşılan bir depo nesnesini birleştirerek Etkinlik/Bilet Sisteminin ilk gerçek sunucusunu çalışır hâle getirdiniz — artık iki farklı `curl` çağrısı aynı veriyi görüyor. Bugün kurduğunuz venv+pip+uvicorn iş akışı, dönem boyunca her Python sunucu projesinde (Hafta 8, Hafta 10, Hafta 13...) aynen tekrarlanacak; `Depends(get_depo)` deseni Hafta 8'de `Depends(get_db_session)`'a, Hafta 10'da `Depends(get_mevcut_kullanici)`'ye genişleyecek — rota fonksiyonlarının bir bağımlılık parametresi alma biçimi değişmeyecek, yalnızca bağımlılığın ne olduğu değişecek. Bugün yazdığınız `HTTPException`/durum kodu disiplini Hafta 10'un "yanlış şifre" `401`'inde, Hafta 11'in yetkisiz erişim `403`'ünde aynı mekanizmayla tekrar karşınıza çıkacak. Hafta 7'de bu sunucuya hiç dokunmadan — yalnızca `fetch` ile ondan veri çekerek — Hafta 5'in ön yüzünü buraya bağlayacaksınız; bugünkü bellekte-depo sınırı ise Hafta 8'de SQLAlchemy ile gerçek bir veritabanına taşınacaktır.